



UNIVERSIDAD TECNOLÓGICA DE PANAMÁ
FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES
DEPARTAMENTO DE INGENIERIA DE SOFTWARE
DESARROLLO DE SOFTWARE PARA PLATAFORMA MÓVIL



Investigación #2

Facilitador: Jorge Cisneros

Tema:

Introducción a Jetpack Compose

Integrantes:

Camaño, Edward	8-1010-515
González, Emanuel	8-1022-2376
Hou, Edwin	8-1021-1916
Lezcano, Fernando	8-1013-1857

Grupo: 1SF241

I SEMESTRE, 2026

Índice

Introducción	3
¿Qué es Jetpack Compose y por qué usarlo?	4
Paradigma declarativo vs. Imperativo	4
Razones técnicas para adoptarlo	4
Arquitectura interna: el compilador de Compose	5
Comparación con XML tradicional	5
Ejemplo comparativo directo	6
Composables básicos de Jetpack Compose	7
Text	7
Button	7
Column y Row	8
Modifier	9
Primer proyecto: UI declarativa completa	9
Dependencias Gradle (build.gradle.kts)	9
Modelo de datos	10
ViewModel con estado	11
Composables de la pantalla	12
TaskScreen — Composable raíz	12
TaskInput — Campo de entrada	13
TaskItem — Tarjeta de tarea individual	14
Punto de entrada: MainActivity	14
Conclusión	17

Introducción

El desarrollo de interfaces de usuario en Android atravesó durante más de una década un camino marcado por la verbosidad, la separación forzada entre lógica y diseño, y una arquitectura de vistas que acumuló deuda técnica con el paso del tiempo. XML definía la estructura visual, mientras que Kotlin o Java se encargaban de manipularla manualmente: un modelo que funcionaba, pero que exigía un esfuerzo desproporcionado para tareas relativamente simples.

Jetpack Compose llega como la respuesta oficial de Google a ese problema estructural. No es una actualización incremental del sistema existente, sino un cambio de paradigma completo: pasar del modelo imperativo donde el desarrollador instruye a la UI cómo cambiar al modelo declarativo, donde simplemente se describe qué debe mostrar la pantalla dado un estado concreto. Este enfoque, que frameworks como React o SwiftUI ya habían popularizado en otras plataformas, finalmente encontró su forma nativa en Android a partir de 2021.

La presente investigación explora Jetpack Compose desde sus fundamentos técnicos hasta su aplicación práctica. Se analiza el modelo mental que propone, su arquitectura interna, los componentes esenciales para construir cualquier interfaz, el sistema de gestión de estado que lo hace reactivo, y finalmente se construye un proyecto funcional que integra todos estos conceptos. El objetivo no es solo aprender una nueva API, sino comprender el razonamiento detrás de cada decisión de diseño del framework, de modo que el conocimiento adquirido trascienda la versión actual y sienta bases sólidas para el desarrollo Android moderno.

¿Qué es Jetpack Compose y por qué usarlo?

Es un kit de herramientas moderno diseñado para simplificar el desarrollo de IU. Combina un modelo de programación reactivo con la concisión y facilidad de uso del lenguaje de programación Kotlin. Es totalmente declarativo, lo que significa que describes tu IU si llamas a una serie de funciones que transforman datos en una jerarquía de IU. Cuando los datos subyacentes cambian, el framework vuelve a ejecutar estas funciones automáticamente y actualiza la jerarquía de la IU por ti.

Actualmente se mantiene como la forma oficial recomendada para el desarrollo de UI en la plataforma Android. Está escrito íntegramente en Kotlin y se integra de forma nativa con el lenguaje, aprovechando sus características como lambdas, extensiones y corrutinas.

Paradigma declarativo vs. Imperativo

En el modelo imperativo el utilizado durante más de una década con Views y XML en Android el desarrollador describe cómo mutar la interfaz en función del estado.

Esto requiere obtener referencias a los widgets (`findViewById`), modificar propiedades manualmente y sincronizar manualmente el estado de la aplicación con la pantalla.

En el paradigma declarativo, el desarrollador describe qué debe mostrar la UI dado un estado concreto. Cuando el estado cambia, el framework recompone automáticamente únicamente los componentes afectados, sin necesidad de instrucciones manuales de actualización.

```
// Imperativo (View system)
val textView = findViewById<TextView>(R.id.label)
textView.text = "Hola"
textView.setTextColor(Color.RED)

// Declarativo (Compose)
@Composable
fun Greeting(name: String) {
    Text(text = "Hola, $name", color = Color.Red)
}
```

Razones técnicas para adoptarlo

- Compose resuelve problemas estructurales del sistema de Views que se habían acumulado durante más de 10 años:
- Eliminación de la jerarquía de vistas compleja: No existen ViewGroup anidados con múltiples passes de medición/layout, lo que reduce la complejidad de renderizado.

- Menos código boilerplate: Desaparece la necesidad de adapters, ViewHolders, LayoutInflaters y bindings XML.
- Interoperabilidad garantizada: Compose puede coexistir con el sistema de Views mediante ComposeView y AndroidView.
- Preview en tiempo real: La anotación `@Preview` permite visualizar composables directamente en Android Studio sin ejecutar el emulador.
- Animaciones de alto nivel: La API de animación de Compose (`animate*AsState`, `AnimatedVisibility`, `Crossfade`) es más expresiva y segura que el sistema de Animators.
- Testing más simple: Los composables son funciones puras (dado el mismo estado producen la misma UI), lo cual simplifica radicalmente el testing.

Arquitectura interna: el compilador de Compose

El compilador de Compose (un plugin del compilador de Kotlin) transforma las funciones `@Composable` en código especial que el runtime de Compose puede rastrear, comparar y redibujar eficientemente. Los mecanismos clave son:

- Recomposición inteligente: Compose recuerda qué partes del árbol de UI dependen de qué piezas de estado. Cuando un estado cambia, solo se recomponen los composables afectados.
- Gap Buffer: El runtime mantiene una estructura de datos tipo slot table para recordar el estado de cada composable entre recomposiciones.
- Estabilidad de tipos: El compilador marca automáticamente los tipos como estables o inestables. Los composables con parámetros estables pueden saltarse la recomposición si sus inputs no cambiaron (skippability).

Comparación con XML tradicional

Para comprender el impacto de Compose es fundamental comparar su modelo mental, rendimiento y mantenibilidad respecto al sistema clásico basado en XML + View. A continuación, se presenta una tabla comparativa técnica:

Aspecto	XML + View System	Jetpack Compose
Lenguaje de UI	XML (declarativo de layout) + Java/Kotlin (lógica)	100% Kotlin, un solo lenguaje
Paradigma	Imperativo: se muta la UI manualmente	Declarativo: se describe el estado deseado
Inflado de layouts	LayoutInflater lee XML en tiempo de ejecución → overhead de reflexión	Sin inflado; el compilador genera código optimizado en tiempo de compilación

Jerarquía de medición	Múltiples passes (measure, layout, draw) en árbol de Views	Un único pass de medición por composable; restricción de un solo paso enforced por el compilador
Gestión de estado	Manual: LiveData + observers + notifyDataSetChanged	Automática: remember + mutableStateOf → recomposición reactiva
Testing	Espresso (lento, frágil, requiere emulador)	ComposeTestRule: testing semántico rápido, sin emulador para unit tests
Preview	Layout editor (limitado, no ejecuta lógica)	@Preview: previewable con datos de muestra reales, múltiples configuraciones
Reusabilidad	Custom Views o fragments (verbosos, difíciles de componer)	Composables: funciones Kotlin ordinarias, altamente componibles
Interop	N/A	ComposeView (Compose dentro de View) y AndroidView (View dentro de Compose)

Ejemplo comparativo directo

El siguiente código muestra cómo construir un contador simple en ambos sistemas para ilustrar la diferencia de verbosidad y modelo mental:

Con XML + ViewModel (sistema clásico):

```

<!-- res/layout/activity_main.xml -->
<LinearLayout ...>
    <TextView android:id="@+id/tvCount" ... />
    <Button android:id="@+id/btnIncrement" android:text="+" ... />
</LinearLayout>

// MainActivity.kt
class MainActivity : AppCompatActivity() {
    private val vm: CounterViewModel by viewModels()
    override fun onCreate(...) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        vm.count.observe(this) { tvCount.text = it.toString() }
        btnIncrement.setOnClickListener { vm.increment() }
    }
}

```

Con Jetpack Compose:

```

@Composable

```

```
fun Counter() {
    var count by remember { mutableStateOf(0) }
    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Text(text = count.toString(), style =
MaterialTheme.typography.headlineLarge)
        Button(onClick = { count++ }) { Text("+") }
    }
}
```

Composables básicos de Jetpack Compose

Un Composable es una función Kotlin anotada con `@Composable` que emite elementos de UI. No retorna un valor, sino que describe la estructura de la pantalla.

El runtime de Compose ejecuta y rastrea estas funciones para construir y actualizar el árbol de nodos UI (internamente llamado `LayoutNode`).

Text

`Text` es el composable fundamental para mostrar texto. Acepta `AnnotatedString` (para texto con estilos mixtos) o `String` plana. Sus parámetros más relevantes:

```
Text(
    text          = "Hola Compose",
    style          = MaterialTheme.typography.bodyLarge, // TextStyle
    color          = MaterialTheme.colorScheme.onBackground,
    fontSize       = 16.sp, // Unidad escalable, respeta ally
    fontWeight     = FontWeight.Bold,
    maxLines       = 2,
    overflow       = TextOverflow.Ellipsis,
    textAlign      = TextAlign.Center,
    modifier       = Modifier.padding(8.dp)
)
```

`AnnotatedString` permite mezclar estilos dentro de un mismo texto:

```
val anotado = buildAnnotatedString {
    append("Texto ")
    withStyle(SpanStyle(color = Color.Red, fontWeight = FontWeight.Bold)) {
        append("rojo y negrita")
    }
    append(" y texto normal.")
}
Text(text = anotado)
```

Button

`Button` implementa el componente de acción principal según Material Design 3. Su slot content: `@Composable RowScope() -> Unit` permite colocar cualquier composable como contenido, aunque lo más común es `Text` o un `Row` con `Icon` + `Text`:

```
Button(
    onClick = { /* acción */ },
    enabled = true,
    shape    = RoundedCornerShape(8.dp),
```

```

        colors = ButtonDefaults.buttonColors(
            containerColor = MaterialTheme.colorScheme.primary
        ),
        modifier = Modifier.fillMaxWidth()
    ) {
        Icon(imageVector = Icons.Default.Add, contentDescription = null)
        Spacer(Modifier.width(8.dp))
        Text("Agregar elemento")
    }

```

Column y Row

Column y Row son contenedores de layout que organizan sus hijos de forma vertical y horizontal respectivamente. Internamente son equivalentes a LinearLayout con orientación vertical/horizontal, pero implementados sin herencia ni reflexión.

```

Column(
    modifier = Modifier.fillMaxSize().padding(16.dp),
    verticalArrangement = Arrangement.spacedBy(12.dp), // espacio entre
    hijos
    horizontalAlignment = Alignment.CenterHorizontally
) {
    Text("Elemento 1")
    Text("Elemento 2")
    Text("Elemento 3")
}

Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceBetween,
    verticalAlignment = Alignment.CenterVertically
) {
    Text("Izquierda")
    Text("Derecha")
}

```

Parámetros clave de Arrangement:

Arrangement	Comportamiento
Arrangement.Top / Start	Agrupar hijos al inicio del eje
Arrangement.Center	Centra todos los hijos en el eje
Arrangement.Bottom / End	Agrupar hijos al final del eje
Arrangement.SpaceBetween	Espacio igual entre hijos; sin espacio en los extremos
Arrangement.SpaceAround	Espacio igual alrededor de cada hijo
Arrangement.SpaceEvenly	Espacio exactamente igual entre y alrededor de los hijos
Arrangement.spacedBy(Xdp)	Espacio fijo definido por el desarrollador entre hijos

Modifier

Modifier es el mecanismo universal de Compose para aplicar transformaciones, restricciones, accesibilidad, eventos y decoraciones a cualquier composible. Es una lista encadenada de instrucciones que se aplican de izquierda a derecha:

```
Modifier
    .fillMaxWidth()           // ocupa todo el ancho disponible
    .height(56.dp)            // altura fija
    .padding(horizontal = 16.dp, vertical = 8.dp)
    .background(Color.Blue, shape = RoundedCornerShape(8.dp))
    .clip(RoundedCornerShape(8.dp)) // recorte para contenido
    .clickable { /* onClick */ }
    .semantics { contentDescription = "Botón de acción" } // accesibilidad
```

Primer proyecto: UI declarativa completa

A continuación, se construye paso a paso una pantalla de lista de tareas (To-Do List) completamente funcional. Esta pantalla permite agregar tareas, marcarlas como completadas y eliminarlas. Cada subsección explica primero qué se está construyendo, por qué se toman las decisiones de diseño, y luego muestra el código correspondiente.

El proyecto integra todos los conceptos estudiados: composables básicos, estado reactivo, state hoisting, ViewModel y Material Design 3, siguiendo un flujo de datos unidireccional.

Dependencias Gradle (build.gradle.kts)

Antes de escribir cualquier composible, es necesario configurar el módulo :app para habilitar Compose y declarar las librerías requeridas. Compose no viene activado por defecto en un proyecto Android; requiere configuración explícita en dos lugares: el bloque dependencies y el bloque android.

La pieza central es el Compose BOM (Bill of Materials). Un BOM es un artefacto especial que actúa como tabla maestra de versiones: al importarlo con platform(), todas las dependencias de Compose quedan sincronizadas automáticamente, eliminando los conflictos de compatibilidad que surgirían al mezclar versiones incompatibles manualmente.

- androidx.compose.ui:ui — Motor central: renderizado, sistema de layout, medición, eventos táctiles y Modifier.
- androidx.compose.material3:material3 — Librería de componentes Material Design 3: Button, Card, Scaffold, TopAppBar, TextField, Checkbox, etc.
- ui-tooling-preview — Habilita la anotación @Preview en Android Studio para visualizar composibles sin ejecutar la app. Solo se incluye en builds de debug.
- activity-compose — Provee la extensión setContent { } en ComponentActivity para iniciar el árbol de Compose desde una Activity.

- **lifecycle-viewmodel-compose** — Expone la funcion `viewModel()` que obtiene o crea un `ViewModel` con scope correcto dentro de cualquier composable.
- **lifecycle-runtime-compose** — Provee `collectAsStateWithLifecycle()`, que colecta un `Flow` respetando el ciclo de vida: pausa la coleccion cuando la app va a segundo plano y la reanuda al volver.

```
// build.gradle.kts (:app)
dependencies {
    val composeBom = platform("androidx.compose:compose-bom:2024.06.00")
    implementation(composeBom)
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.material3:material3")
    implementation("androidx.compose.ui:ui-tooling-preview")
    debugImplementation("androidx.compose.ui:ui-tooling")
    implementation("androidx.activity:activity-compose:1.9.0")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.2")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.2")
}

// Habilitar Compose en android { }
android {
    buildFeatures { compose = true }
    composeOptions { kotlinCompilerExtensionVersion = "1.5.14" }
}
```

`buildFeatures { compose = true }` activa el plugin del compilador de Compose. Sin esta linea, las anotaciones `@Composable` no seran procesadas y el proyecto no compilara. La version de `kotlinCompilerExtensionVersion` debe ser compatible con la version de Kotlin del proyecto.

Modelo de datos

El modelo de datos es la representacion de una tarea en memoria. Se define como una `data class` de Kotlin, que por convencion en Compose debe ser inmutable: todos sus campos son `val`, no `var`. La inmutabilidad es esencial porque el sistema de recomposicion de Compose compara objetos por valor (`equals()`) para decidir si un composable debe recomponerse o puede saltarse.

La clase `Task` tiene tres campos: `id` para identificar unicamente cada tarea y poder encontrarla eficientemente en operaciones de `toggle` y `delete`; `title` para el texto que el usuario escribe; y `completed` con valor por defecto `false` para representar si la tarea esta terminada. Al ser una `data class`, Kotlin genera automaticamente `equals()`, `hashCode()` y `copy()`, este ultimo es fundamental para las actualizaciones inmutables del `ViewModel`.

```
data class Task(
    val id      : Int,
    val title   : String,
```

```
val completed: Boolean = false
```

```
)
```

Cada vez que se modifica una tarea (toggle, delete), no se muta el objeto existente sino que se crea uno nuevo con `copy()`. Esto garantiza que Compose detecte el cambio de referencia y reactive la recomposicion unicamente en los composables que dependen de esa tarea.

ViewModel con estado

El ViewModel es la pieza que separa la logica de negocio de la interfaz de usuario. Sobrevive a rotaciones de pantalla y cambios de configuracion porque Android lo mantiene en memoria independientemente del ciclo de vida de la Activity. En este proyecto el ViewModel tiene dos responsabilidades: almacenar el estado actual de la pantalla (TaskUiState) y procesar los eventos del usuario (agregar, completar, eliminar tareas).

TaskUiState es una data class que encapsula todo lo que la pantalla necesita para renderizarse: la lista de tareas existentes y el texto que el usuario ha escrito en el campo de entrada. Agrupar el estado en un unico objeto tiene una ventaja critica: cada actualizacion emite un nuevo objeto al Flow, lo que garantiza consistencia y evita condiciones de carrera.

El estado interno se almacena en un MutableStateFlow privado y se expone como StateFlow de solo lectura. Este patron de encapsulacion evita que la UI modifique el estado directamente, obligando a que toda mutacion pase por los metodos del ViewModel, lo que hace el flujo de datos predecible y testeable.

- `onInputChange` — Actualiza el texto del campo de entrada mientras el usuario escribe, usando `update { it.copy(...) }` para producir un nuevo estado inmutable.
- `addTask` — Valida que el texto no este vacio, crea un nuevo Task con un ID basado en el timestamp actual, lo agrega a la lista y limpia el campo de entrada en una sola operacion atomica.
- `toggleTask` — Busca la tarea por ID en la lista y crea una copia con `completed` invertido. Las demas tareas permanecen sin cambio (else task).
- `deleteTask` — Filtra la lista excluyendo la tarea con el ID indicado, produciendo una nueva lista sin ella.

```
data class TaskUiState(  
    val tasks      : List<Task>  = emptyList(),  
    val inputText  : String      = ""  
)  
  
class TaskViewModel : ViewModel() {  
    private val _uiState = MutableStateFlow(TaskUiState())  
    val uiState: StateFlow<TaskUiState> = _uiState.asStateFlow()  
  
    fun onInputChange(text: String) {  
        _uiState.update { it.copy(inputText = text) }  
    }  
}
```

```

    }

    fun addTask() {
        val text = _uiState.value.inputText.trim()
        if (text.isEmpty()) return
        val newTask = Task(id = System.currentTimeMillis().toInt(), title =
text)
        _uiState.update { it.copy(tasks = it.tasks + newTask, inputText =
"" ) }
    }

    fun toggleTask(id: Int) {
        _uiState.update { state ->
            state.copy(tasks = state.tasks.map { task ->
                if (task.id == id) task.copy(completed = !task.completed)
else task
            })
        }
    }

    fun deleteTask(id: Int) {
        _uiState.update { it.copy(tasks = it.tasks.filter { t -> t.id != id
}) }
    }
}

```

Composables de la pantalla

La pantalla se divide en tres composables con responsabilidades bien delimitadas. Esta separación sigue el principio de responsabilidad única y aplica state hoisting: cada composable hijo recibe lo que necesita como parámetro y delega las mutaciones al padre mediante callbacks.

TaskScreen — Composable raíz

TaskScreen es el composable de nivel superior. Es el único que conoce al ViewModel y, por tanto, el único punto donde el estado se colecta con `collectAsStateWithLifecycle()`. Todos los composables hijos reciben datos ya extraídos del estado, sin acceso directo al ViewModel. Esta es la aplicación del patrón state hoisting a escala de pantalla.

Scaffold es el composable de layout estructural de Material 3. Gestiona automáticamente los slots de TopAppBar, BottomBar, FAB y el contenido principal, calculando el padding interior correcto para que el contenido no quede debajo de la barra de estado o de navegación del sistema. El lambda `{ padding -> }` recibe ese padding y debe aplicarse obligatoriamente al contenido.

Dentro del Scaffold, un Column organiza verticalmente el campo de entrada y la lista. LazyColumn es el equivalente eficiente de RecyclerView en Compose: solo renderiza los items visibles en pantalla, descartando los que están fuera del viewport. El parámetro `key = { it.id }` es crítico: permite a Compose rastrear la identidad de cada item entre recomposiciones, habilitando animaciones de inserción/eliminación y evitando recomposiciones innecesarias.

```

@Composable
fun TaskScreen(vm: TaskViewModel = viewModel()) {
    val state by vm.uiState.collectAsStateWithLifecycle()
}

```

```

Scaffold(
    topBar = {
        TopAppBar(title = { Text("Mis Tareas") })
    }
) { padding ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(padding)
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        TaskInput(
            value = state.inputText,
            onValueChange = vm::onInputChange,
            onAddTask = vm::addTask
        )
        LazyColumn(verticalArrangement = Arrangement.spacedBy(8.dp)) {
            items(state.tasks, key = { it.id }) { task ->
                TaskItem(
                    task = task,
                    onToggle = { vm.toggleTask(task.id) },
                    onDelete = { vm.deleteTask(task.id) }
                )
            }
        }
    }
}

```

TaskInput — Campo de entrada

TaskInput es un composable completamente stateless: no posee ningun estado interno. Recibe el valor actual del campo (value), un callback para notificar cambios (onValueChange) y otro para confirmar el envio (onAddTask). Esta firma es identica al patron que usan los composables de Material 3 como OutlinedTextField, lo que facilita la integracion y el testing.

OutlinedTextField implementa el estilo de campo de texto con borde de Material Design 3. El parametro Modifier.weight(1f) hace que el campo ocupe todo el espacio horizontal disponible dentro del Row, dejando al Button su ancho natural. keyboardOptions con ImeAction.Done muestra el boton de confirmacion en el teclado virtual; keyboardActions lo enlaza con la accion de agregar la tarea, lo que evita que el usuario tenga que tocar el boton en pantalla.

```

@Composable
fun TaskInput(
    value : String,
    onValueChange: (String) -> Unit,
    onAddTask : () -> Unit
) {
    Row(verticalAlignment = Alignment.CenterVertically) {
        OutlinedTextField(
            value = value,
            onValueChange = onValueChange,
            modifier = Modifier.weight(1f),
            placeholder = { Text("Nueva tarea...") },
            singleLine = true,
            keyboardOptions = KeyboardOptions(imeAction = ImeAction.Done),
            keyboardActions = KeyboardActions(onDone = { onAddTask() })
        )
        Spacer(Modifier.width(8.dp))
    }
}

```

```

        Button(onClick = onAddTask) { Text("Agregar") }
    }
}

```

TaskItem — Tarjeta de tarea individual

TaskItem representa visualmente una sola tarea. Recibe el objeto Task y dos callbacks: onToggle para cambiar el estado completado y onDelete para eliminarla. Al igual que TaskInput, no tiene estado propio: es una función pura que mapea un Task a una representación visual.

Card provee la superficie elevada característica de Material Design 3, con esquinas redondeadas y sombra configurable mediante elevation. Internamente, un Row con SpaceBetween separa el lado izquierdo (Checkbox + texto) del icono de eliminar. El texto de la tarea cambia visualmente según su estado: si está completada se aplica TextDecoration.LineThrough (tachado) y el color cambia a gris, comunicando el estado al usuario sin necesidad de palabras adicionales.

```

@Composable
fun TaskItem(task: Task, onToggle: () -> Unit, onDelete: () -> Unit) {
    Card(
        modifier = Modifier.fillMaxWidth(),
        elevation = CardDefaults.cardElevation(defaultElevation = 2.dp)
    ) {
        Row(
            modifier = Modifier.padding(12.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Checkbox(checked = task.completed, onCheckedChange = {
                    onToggle()
                })
                Spacer(modifier = Modifier.width(8.dp))
                Text(
                    text = task.title,
                    style = if (task.completed)
                        LocalTextStyle.current.copy(textDecoration =
                            TextDecoration.LineThrough)
                        else LocalTextStyle.current,
                    color = if (task.completed) Color.Gray else
                        Color.Unspecified
                )
            }
            IconButton(onClick = onDelete) {
                Icon(Icons.Default.Delete, contentDescription = "Eliminar
                tarea")
            }
        }
    }
}

```

Punto de entrada: MainActivity

MainActivity es el único punto de entrada Android de la aplicación. Su única responsabilidad en este proyecto es iniciar el árbol de Compose mediante setContent {}. Esta función de extensión, provista por activity-compose, reemplaza completamente a setContentView(R.layout.activity_main) del sistema tradicional.

Dentro de setContent se aplica MaterialTheme, que inyecta el sistema de tokens de Material Design 3 (colores, tipografía, formas) en todo el árbol de composables descendientes mediante CompositionLocal. Cualquier composable dentro de este árbol puede acceder a

MaterialTheme.colorScheme, MaterialTheme.typography y MaterialTheme.shapes. Surface establece el color de fondo correcto segun el tema activo (claro u oscuro) y actua como la raiz visual del contenido.

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                Surface(color = MaterialTheme.colorScheme.background) {
                    TaskScreen()
                }
            }
        }
    }
}
```

ComponentActivity es la clase base recomendada (parte de androidx.activity). Reemplaza a AppCompatActivity cuando se usa Compose de forma exclusiva, aunque tambien es compatible con el sistema de Views si se necesita interoperabilidad.

Flujo de datos unidireccional (UDF)

La arquitectura del proyecto sigue estrictamente el patron UDF: los eventos fluyen hacia arriba (del composable al ViewModel mediante callbacks) y el estado fluye hacia abajo (del ViewModel al composable mediante StateFlow). Este patron garantiza que la UI sea siempre una funcion determinista del estado: dado el mismo TaskUiState, la pantalla se verá siempre identica, sin importar como se llegó a ese estado.

El beneficio práctico del UDF es que hace el codigo predecible y testeable: para probar addTask() basta con llamar al metodo del ViewModel y verificar que el nuevo TaskUiState contiene la tarea esperada, sin necesidad de instanciar ninguna Activity ni Composable.

Capa	Responsabilidad	Tecnología
UI (Composables)	Renderizar estado; emitir eventos de usuario	Jetpack Compose
ViewModel	Procesar eventos; actualizar UiState	ViewModel + StateFlow
Repository	Fuente de verdad de los datos	Room / Retrofit / DataStore

La capa de Repository no se implementa en este ejemplo introductorio. En producción, el ViewModel delegaría la persistencia de tareas a un repositorio respaldado por Room (SQLite) o un backend remoto.

Conclusión

Jetpack Compose representa un cambio real en la forma de construir interfaces Android, no solo una actualización de herramientas. A lo largo de esta investigación quedó claro que su valor no está únicamente en escribir menos código, sino en adoptar un modelo mental más coherente: la UI como función del estado, la recomposición como mecanismo automático y la inmutabilidad como contrato que hace el sistema predecible.

El proyecto de lista de tareas lo ilustra de forma concreta: sin un solo archivo XML, sin adapters ni observers manuales, se construyó una pantalla funcional, reactiva y con una arquitectura limpia. Esa proporción entre esfuerzo y resultado es precisamente lo que Compose ofrece cuando se entienden bien sus fundamentos.

Quedan temas por explorar animaciones, navegación, testing, side effects pero todos son extensiones naturales de los mismos principios estudiados aquí. Aprender Compose hoy no es solo mantenerse actualizado; es prepararse para escribir el desarrollo Android de los próximos años con bases sólidas.

Bibliografía

Google. (2024). *Jetpack Compose — Android Developers*. Android Developers. <https://developer.android.com/jetpack/compose>

Google. (2024). *Compose BOM version mapping*. Android Developers. <https://developer.android.com/jetpack/compose/bom/bom-mapping>

Google. (2024). *State and Jetpack Compose*. Android Developers. <https://developer.android.com/jetpack/compose/state>

Google. (2024). *Thinking in Compose*. Android Developers. <https://developer.android.com/jetpack/compose/mental-model>

Google. (2024). *Compose layout basics*. Android Developers. <https://developer.android.com/jetpack/compose/layouts/basics>

Google. (2024). *Material Design 3 in Compose*. Android Developers. <https://developer.android.com/jetpack/compose/designsystems/material3>

Google. (2024). *ViewModel and State in Compose*. Android Developers. <https://developer.android.com/topic/libraries/architecture/viewmodel>

Google. (2024). *Kotlin compiler compatibility map — Compose*. AndroidX Releases. <https://developer.android.com/jetpack/androidx/releases/compose-kotlin>

Google. (2024). *Now in Android — Compose Samples*. GitHub. <https://github.com/android/nowinandroid>

Google. (2024). *Compose Samples*. GitHub. <https://github.com/android/compose-samples>